EE4292 IC Design Laboratory, Fall 2025
Final Project
Hardware-Efficient Real-Time Color Transfer Engine

楊明諺, *EE, NTHU*, 黃威儒, *EE, NTHU*, and 吳冠廷, *EE, NTHU*

***Abstract* - 本專案實作 1080p/60FPS 即時色彩轉移引擎。RAW 影格前處理後轉 YCbCr，Y 做亮度匹配、CbCr 做色度映射；以雙 FSM 與 SRAM 分割並行。最終達成 7.611ms/Frame 並通過驗證。**

I. Introduction

本次專案實作一套硬體友善的即時色彩轉移引擎，目標為建立一條從原始影像資料到與參考圖片色彩風格一致的完整影像 RGB 輸出處理流程。系統輸入為數位相機所產生之解析度 1920*1080 的 DNG 原始影格，其內容為 RGGB Bayer pattern 的 RAW 資料型態。相較於直接以 RGB 影像作為輸入，從 Linear domain (RAW) 開始處理能更早修正感光元件通道不平衡與光源造成的色偏問題，使後續整體色彩增強結果更一致。透過獲取參考圖片的亮度分布與彩度統計資料，可以轉換原始影像的色彩風格，就像將原始影格套上一層客制化濾鏡後輸出。此外，我們目標在 16 ms 內處理完一個影格，即時處理 60 FPS 的影像而不掉幀，發揮特定應用晶片在演算法運算速度上的優勢。本系統採用定點數運算。先以 Python 建立軟體模型並作為 RTL 驗證的 golden output，以確保功能正確性與硬體可行性。再透過 EDA 工具合成、驗證，最後合成出完整晶片 layout。

Fig. 1 Illustration of this Color Transfer engine

本專案之所有演算法皆是為了這堂課程的 final project 設計的，所有內容都是在這個課程期間所產出。有依據我們所需演算法和 AI 協作，產生 Python 軟體程式碼，不過這段程式碼也經過多次人工修改才成為最後的 golden model；用 AI 產生除法器部分程式碼，其餘內容皆為人工製作。

II. Algorithm & Implementation

整體軟體流程如 Fig.2 所示。分為 RAW 影像前處理與色彩轉移兩個階段。在前處理階段，首先對 RGGB RAW 影像進行白平衡校正，接著利用 demosaic 將 Bayer pattern 插值轉為完整 RGB 影像，並透過 LUT 方式進行 Gamma 校正。後續的色彩增強階段則以統計式方法為核心，先進行 RGB 白平衡正規化，再將影像轉換至 YCbCr 色彩空間，分別對亮度與色度進行獨立處理。亮度通道利用 histogram match改善亮度與對比，色度通道則透過高斯統計匹配調整整體色調與飽和度，最後轉換回RGB輸出。

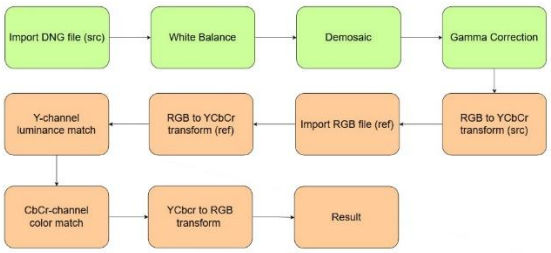


Fig. 2 Flowchart of the software system

在硬體方面，我們將原本的浮點數量化成Q8,8的定點數運算。由於輸入圖片有二，分別為 source (RAW 檔案輸入) 與 reference (RGB 檔案輸入)，而兩者在硬體上可以並行計算，如 Fig. 3 所示。因此在 top module 中，我們分別設計了 src 與 ref 兩個 FSM 去控制所有 module，在 src 進行 RAW 檔案轉換時， ref 可以先行從 RGB 轉換為 YCbCr 並進行全域的 CbCr 統計。除此之外在色彩轉移時，我們讓 Y 通道的 light-matching 和 CbCr 的 color-mapping 同時運算，增加整體並行度。

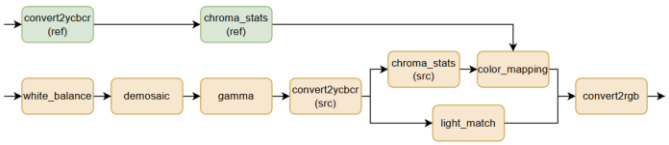


Fig. 3 Execution flow of hardware system

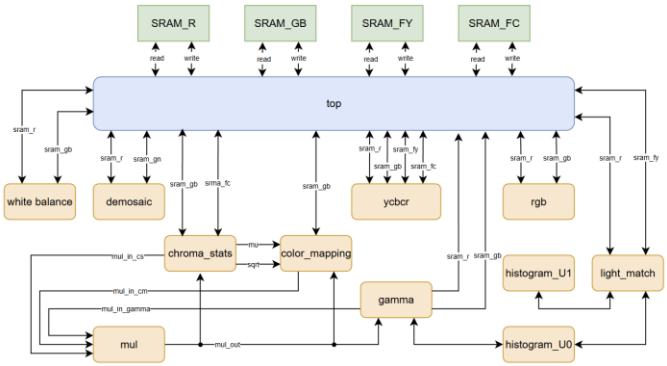


Fig. 4 Block diagram of the hardware system

Fig. 4 為本系統的 Block Diagram。Top module 連接對外的四個 SRAM ，並連接對內所有 SRAM next 訊號，如此一來所有的 module 可以共用一組 buffer ，並由兩個 FSM 決定所有 signal。

以下介紹重點步驟的詳細演算法與硬體實作細節。

A. *SRAM Design*

本次 project 實作自適應色彩增強系統，其中 src 與 ref 圖片大小皆為 1920 * 1080 pixel，格式分別為 RGGB (RAW) 與 RGB。由於圖片 pixel 數量眾多 (~2.07M)，為兼顧效能與硬體資源用量，我們以一個位址 4*4 個 pixel 為單位，並切分為 4 個bank，分別存放 4 個 pixel的資料。Fig. 5 顯示任一個 4*4 的 raw data input 的存放方式。

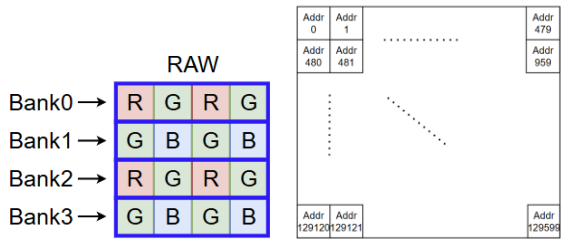


Fig. 5, 6 address 中 pixel 存放方式 和 整張圖片 address 排列方式

考量到輸入圖片有 src 與 ref 兩種，同時有 Y 跟 CbCr 並行運算的需求，因此我們總共設計 4 個 SRAM 以滿足平行化與加速需求，分別為SRAM_R, SRAM_GB, SRAM_FY, SRAM_FC。其中 SRAM_R 負責儲存 source 的 R-channel 以及轉成 YCbCr 後的Y-Channel；SRAM_GB 則是儲存 source 的 G-channel 與 B-Channel，以及轉成 YCbCr 後的 Cb-Channel 與 Cr-Channel。SRAM_FY與SRAM_FC 的功能相同，只是改為儲存reference。

在初始輸入時，因為 RAW 檔 RGGB 的格式 Channel info 有缺失，因此我們將初始的輸入跳著存入 SRAM 中，如 Fig. 6 所示，如果該 pixel 的 channel 有缺失，則初始值會存入 0，module 在讀值的時候也不會 access 該 bit 數的值，如此便可以在相同架構下重複使用這四個 SRAM。除此之外，輸入 linear domain (16 bit per channel) 與後續的 gamma domain - RGB (8 bit per channel)、color domain-YCbCr (Q8, 8 bit per channel)，由於不同階段資料的儲存型態不同，因此在 SRAM 設計上都採用 16 bit per channel 並共用相同 SRAM，大幅減少 SRAM 用量。

關於 SRAM 每個 bank 的儲存方式，由於我們儲存格式為定點數 Q8,8，因此 SRAM_R 每 bank 為 64 bits (4 個 pixel * 16 bits)，SRAM_GB與SRAM_FC 每個 bank 為 128 bits (8 個 pixel * 16 bits)，SRAM_FY 每個 bank 為 32 bits (4 個 pixel * 8 bits，因為reference 是 RGB 格式)。

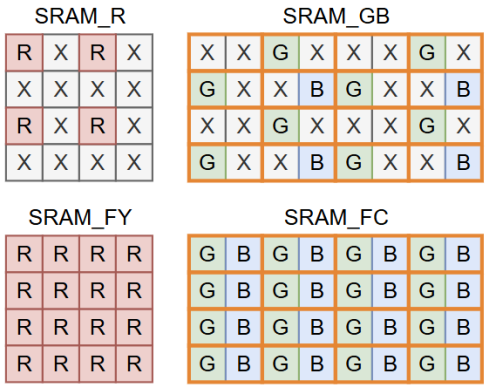


Fig. 7 Input data storage method at SRAM

B. *White Balance*

白平衡 (White Balance) 目的在於補償不同光源造成的色偏，讓照片中的白色物體能準確還原為白色，使整體色彩看起來自然。我們的作法是先統計影像中 R、G、B 三個通道的平均值，分別記為 μ_R, μ_G, μ_B。接著以綠色通道作為基準，計算紅色與藍色的增益（gain）

$$g_R = \frac{\mu_G}{\mu_R}, \quad g_G = 0.890625, \quad g_B = \frac{\mu_G}{\mu_B} \tag{1}$$

由於人眼對綠色較為敏感，因此我們對綠色通道乘上一個 gain $g_G = 0.890625$。最後將像素分別乘上對應的增益即可完成白平衡校正：

$$R' = g_R \cdot R, \qquad G' = g_G \cdot G, \qquad B' = g_B \cdot B \tag{2}$$

這部分需要遍歷整張圖片兩次，會耗費大量 cycle，所以採取每讀入一筆 address 即對整個 address 做運算，以此減少 cycle 數。同時對累加運算做 pipeline 以壓低 timing。

C. *Demosaic*

去馬賽克 (Demosaic) 的用意是將原先在 Linear Domain (RGGB) 的圖片轉換為 Linear RGB domain (RGB)。我們的作法是採用 Bilinear interpolation 的方法，利用鄰近像素計算平均填補缺失的 channel。

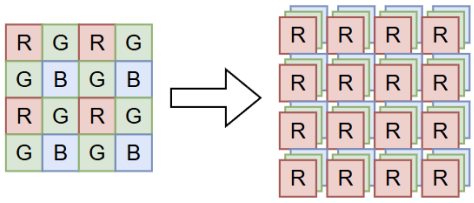


Fig. 8 RGGB to RGB conversion

在像素的重建上，不同位置有以下的重建方式：

R位置：G 以四方向相鄰平均，B 以四對角相鄰平均

G位置：R、B 以上下或左右方向平均

B位置：G 以四方向相鄰平均，R 以四對角相鄰平均

硬體實作上，為了減少 SRAM access 次數並提升 throughput，在設計時採用 window 與 shift register 的結構，window大小為 8*9 個 pixel，需要 access 6 次 SRAM 方能填滿整個 window，每次從 SRAM 讀取 4 個 bank 值並寫入 buffer window 中，當 window prefill 到一定程度後，便可以在 access SRAM 的同時，讀取 window 上固定位置的 channel 值，並計算出缺失的 RGB channel 值。

D. Gamma Correction

伽碼校正 (Gamma correction) 的目的，是把線性光強轉換成符合人眼感知的亮度表示。做法通常是以冪次函數做非線性映射:

$$I_{out} = I_{in}^{1/\gamma} \tag{3}$$

做完之後的圖片就會從 Linear RGB domain 轉換至 Gamma Domain (RGB)。

為降低硬體實作複雜度，我們對傳統 Gamma 校正流程進行簡化。首先，我們以 white point 作為亮度基準，將 RGB 三個通道的像素值正規化至區間 [0, 1]。White point 的取得方式為：對各通道建立 256-bin 直方圖，並由低亮度端累加像素數，當累積達總像素數的 96% 時，對應之 bin 定義為 white point，以避免少量過曝像素影響整體尺度。此外，在建立直方圖時採用取樣方式 (sample)，每 20 個像素僅累計一次，進一步提升計算速度並減少面積。

在 Gamma 校正部分，我們捨棄原先的分段式模型，改採單一冪次轉換，其定義為：

$$y = 1.055 * x^{\frac{1.4}{2.2}} - 0.5 \tag{4}$$

其中 x 為正規化後之像素值。冪次指數 $\frac{1.4}{2.2}$ 是經多組實驗比較後選定，可在亮度表現與視覺效果之間取得較佳平衡。實作上，該冪次運算透過查表法（LUT）完成，以提升硬體實作效率。

E. YCbCr/RGB Transform

YCbCr 是一種將影像亮度（Luminance）與色度（Chrominance）分離的色彩表示方式。其中 Y 表示亮度，Cb / Cr 分別表示藍色與紅色方向的色彩偏移 (色度)。RGB 與 YCbCr 之間的轉換方式如下:

$$Y = 0.299R + 0.587G + 0.114B \tag{5}$$
$$Cb = -0.1687R - 0.3313G + 0.5B + 128.0 \tag{6}$$
$$Cr = 0.5R - 0.4187G - 0.0813B + 128.0 \tag{7}$$

$$R = Y + 1.402 * Cr \tag{8}$$
$$G = Y - 0.34414 * Cb - 0.71414 * Cr \tag{9}$$
$$B = Y + 1.772 * Cb \tag{10}$$

F. Luminance Match

亮度匹配 (Luminance Match) 旨在將來源影像（source）的亮度分佈調整為與參考影像（reference）一致，以達到整體明暗風格對齊。我們使用的方式為 Histogram Specification 建立亮度映射函數。

令 $y \in \{0, \dots, 255\}$ 為離散亮度值。對 source 與 reference 影像在 Y 通道分別統計直方圖並累加獲得累積分佈函數 (CDF)，接著令 source 的每一個 CDF 的值對應到 reference 中最接近的 CDF 的值，形成輸入到輸出的查表映射 (LUT)。最後對每個 source 亮度值 y 使用這個 LUT 進行亮度映射。

這部分需要進行全域的亮度統計，以建立 CDF 統計函數，然而進行全域的統計需要遍歷每個 pixel，在硬體執行上並不友善，因此與 gamma 使用的方式一樣，我們改用 sample 的方式對圖片亮度進行統計 (with step = 20)，在 CDF 函數維持相似的統計結果，又大幅減少處理時間。同時在全域統計上和前面的 gamma module 共用 histogram module，減少整體硬體資源使用量。

G. Color Match

色度匹配 (Luminance Match) 是將來源影像（source）的色度分佈調整為與參考影像（reference）一致，以達到整體色彩風格對齊。做法是計算兩個影像色度(Cb, Cr)的平均值 (μ) 及標準差 (σ)，並對 source 的每個像素透過以下公式轉換 x (Cb, Cr) 成 y (Cb', Cr'):

$$y = (x - \mu_{src}) \cdot \frac{\sigma_{ref}}{\sigma_{src}} + \mu_{ref} \tag{11}$$

計算流程是先算出四組 (Cb ref, Cr ref, Cb src, Cr src) 的 μ, σ^2，再遍歷整張圖算出 Cb', Cr'。在前半部計算時，我們選擇適合硬體並行化的標準差計算公式以減少圖片遍歷次數：

$$\sigma = \sqrt{E(x^2) - E^2(x)} \tag{12}$$

由上述公式，在算平均值 $\mu = E(x)$ 時，能同時計算 $E(x^2)$。搭配對平方、累加做 pipeline，可以在低 timing 下只需遍歷一次圖片就算出 μ, σ^2。此外，在 top module 使用第二個 FSM 提前並行地算 reference 圖片的 μ, σ^2，可以重複使用同份硬體，減少硬體閒置。

在後半部的 Cb', Cr' 運算，我們將式(11) 改寫為斜截式，預先計算常數部分，就可以在每次讀取 Cb, Cr 時用一次乘法和一次加法算出 Cb', Cr'：

$$y = x \cdot \frac{\sigma_{ref}}{\sigma_{src}} - \left(\mu_{src} \cdot \frac{\sigma_{ref}}{\sigma_{src}} - \mu_{ref}\right) \tag{13}$$

Color matching 的其他細節為：根號使用 LUT 以減少 cycle 數；定點數使用較小 bit 數能減少根號 LUT、除法器、乘法器的面積。

III. VALIDATION

由於我們在演算法上做了許多近似或是加速方式，如定點數運算、取樣 (sample) 等，因此為驗證我們的硬體實作是否正確，我們在原始軟體的基礎上根據硬體的邏輯額外寫了另一份專門生 golden file 的軟體來驗證我們的模型。

在進行驗證之前，我們首先比較了此 golden model 與原始軟體模型所生成的輸出結果，結果顯示兩者在視覺上無明顯差異，代表在引入上述近似與加速方法後，模型行為仍與原始演算法高度一致。因此我們選擇用這份 golden model 生成 golden file 來比對我們的輸出結果，確定我們硬體模型的正確性。

Fig. 9, 10 原軟體模型(左)與新的 golden 模型生成的圖片(右)

比對方式就是將存在 SRAM_R 與 SRAM_GB 的輸出結果與生成的 golden file 進行逐 address 的比較，全部相同即代表硬體行為是正確的。

IV. RESULT

在原始軟體中，處理一張圖片的時間約為 573.58 ms，而我們在硬體實現中達成 7.611 ms 內處理完一張影格 (~75x 加速)，理想上可以最高 131.4 FPS 的頻率即時轉換原始影像色彩風格。Layout 後的晶片規格如下所列：

Throughput: 131.4 frame/sec @ 153.8 MHz
- Timing: 6.500 ns
- Cycle: 1170861 (cycles/frame)

TABLE I.
AREA AND POWER ANALYSIS OF OUR CHIP

	Pre Layout	Post Layout	
		Pre-sim waveform	Post-sim waveform
Area (μm^2)	459657.586254	613141.362 (utilization: 0.75)	
Net Switching Power (mW)	0.2838	1.499	1.868
Cell internal Power (mW)	2.292	2.338	2.534
Total Power (mW)	2.588	3.852	4.417

A. *APR Layout*

為了增加設計效率，在 APR 部分我們一次並行測試多種 utilization。同時在 timing 上預留些 margin，以利 APR routing 以及 postsim 驗證。

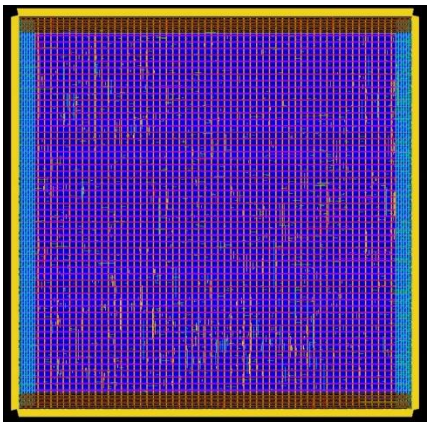


Fig. 11 Layout of our design

B. *Verification*

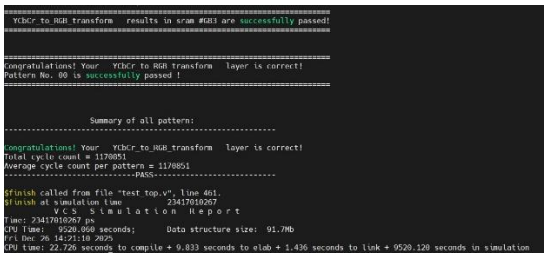
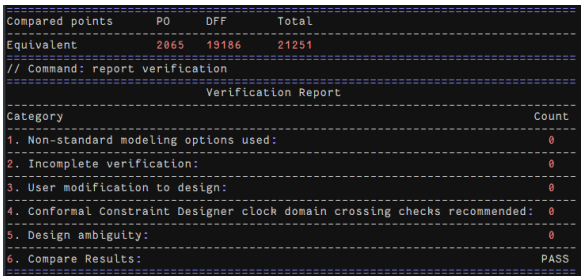


Fig. 12 Post-sim result pass



Compared points	PO	DFF	Total
Equivalent	2065	19186	21251

// Command: report verification

Verification Report

Category	Count
1. Non-standard modeling options used:	0
2. Incomplete verification:	0
3. User modification to design:	0
4. Conformal Constraint Designer clock domain crossing checks recommended:	0
5. Design ambiguity:	0
6. Compare Results:	PASS

Fig. 13 Post-layout LEC equivalent

V. PROBLEM ENCOUNTER

A. *Python 浮點數精度*

起初我們在 Python 有時使用浮點數做運算，但在 RTL 是定點數運算，導致生成的 testbench 無法與 RTL 對齊，會有微小精度差異。例如在做浮點數累加時會與定點數有微小差異，量化時使誤差放大，並持續累積使結果錯誤。解決方法是直接將整個 python 程式碼在整數運算，免除精度誤差。

B. *圖片太大*

在初始設定 target 時，為達到處理 1080p 高解析度的圖片之目標，我們將圖片大小設定為 1080 * 1920 ，光一張圖片的 pixel 數就多達兩百多萬，此舉造成我們在後續不管是 presim 驗證、gatesim、postsim、primetime power 分析，test cycle 數量很大需要耗費很長的時間等待模擬結果，使整體設計、驗證周期拉長，debug 效率大幅下降。

我們折衷的解決辦法是同時使用多台伺服器跑不同設定檔，加快驗證效率。未來可先將圖片縮小，進行整體功能的驗證，最後再將圖片設定置目標大小進行少量 pattern 測試。

VI. Conclusion

本次 project 最終達成單張影格處理時間 7.611 ms，可達 131.4 FPS（153.8 MHz、1,170,861 cycles/frame，timing 6.500 ns），且經驗證後晶片輸出與 Python 軟體端完全對齊。在設計取捨上，我們以取樣式全域統計（例如 histogram / CDF 以 step = 20 取樣）降低運算複雜度，同時透過 reference 端統計前移與雙 FSM 並行設計提升硬體利用率與平行度。過程中，我們克服 Python 浮點與 RTL 定點精度不一致、以及 1080p 影像導致驗證與功耗分析模擬週期過長等問題。在後續的設計中，可朝向更嚴謹的定點精度優化、縮短驗證流程（如分層/抽樣驗證）、以及在 post-layout 下進一步針對功耗與 APR routing margin 做最佳化，以提升整體設計效率與晶片效能。

References

[1] E. Reinhard, M. Adhikhmin, B. Gooch, and P. Shirley, "Color Transfer between Images," IEEE Computer Graphics and Applications, 2001.